

TEAM MEMBER 2

Code Implementation and Numerical Experiments

Practical Execution Guide | What to Do, How to Do It, and What to Deliver

Prerequisite: The team has already completed a high-accuracy reference solution. You do not need to produce it again. All subsequent error, stability, efficiency, and tolerance experiments must use the same team-approved `t_ref` and `y_ref`.

Your core responsibility: Establish what actually happens when the code runs. Implement numerical methods, design and execute experiments, generate data, figures, and tables, record computational costs, and enable others to reproduce everything with a single `run_all` command. The mathematics team is responsible for extended theoretical derivations.

Recommended Execution Order - Follow This Sequence

1. First implement the Robertson RHS, Jacobian, EE, RK4, and IE + Newton correctly, and perform basic unit tests.
2. Next conduct the fixed-step convergence study. First confirm approximately first-order convergence for EE and IE and fourth-order convergence for RK4. If the orders are incorrect, debug before proceeding.
3. Then generate the stability regions, Jacobian eigenvalues, and stiffness ratio.
4. Next perform the Explicit Euler failure sweep, diagnostics, matched-error comparison, and Newton tolerance sweep.
5. Finally implement adaptive step-size control and the cost-vs-accuracy experiment, and use `run_all` to generate all final figures and tables automatically.

1. Standardize the Code Structure and Interfaces

What You Need to Do

Divide the project into clear, small functions. Do not put all the code into a single script. The recommended minimum set is:

- `robertson_rhs`
- `robertson_jacobian`
- `explicit_euler`
- `rk4`
- `implicit_euler`
- `newton_solver`
- `error_diagnostics`
- `stiffness_analysis`
- `adaptive_solver`
- `run_all`

How to Implement It

- Use the same input/output format for EE, RK4, and IE wherever possible, for example `solver(t0, tf, y0, h, options) -> t, Y, stats`. This allows later sweeps to run different methods using the same loop.
- Use a consistent stats structure to store quantities such as `steps`, `rhs_evals`, `newton_iters`, `accepted_steps`, `rejected_steps`, and `converged`. Set fields that do not apply to a method to 0 or NaN.
- Keep all adjustable parameters in a single config or options structure: `h`, Newton tolerance, Newton maxiter, the zero-eigenvalue threshold, the accuracy threshold, the adaptive tolerance, and so on. Do not scatter hard-coded values throughout the code.
- Route all error comparisons through the same `compare_to_reference` or `error_diagnostics` function. If the numerical solution times differ from `t_ref`, the team must agree on a single alignment/interpolation rule. All methods must use that rule, and the report must explain it. Do not let each solver align the data differently.
- Any sanity-check values supplied in the problem may only be used for checking. They must not be embedded in the algorithm logic.

Completion Criteria

- Changing the method only requires changing its name or function handle, without rewriting the experiment scripts.
- `run_all` calls these modules instead of containing multiple copied-and-pasted versions of the code.

2. Implement Explicit Euler (EE)

$$y_{n+1} = y_n + h f(t_n, y_n)$$

How to Implement It

- Inputs: `t0`, `tf`, `y0`, and `h`. Calculate the required number of steps first. If the interval ending at `tf` is not an exact multiple of `h`, the team must agree on and document how to handle the final step.
- Call `robertson_rhs` once per step and update the three-dimensional state vector `y = [y1, y2, y3]^T`.
- Save the full trajectory as `t` and `Y`, with each row of `Y` containing `[y1, y2, y3]` at one time point.
- Increment `rhs_evals` by 1 on every RHS call and `steps` by 1 after every completed step.
- Add failure detection. If NaN/Inf, clear numerical overflow, or a state norm above the team-defined blow-up threshold occurs, mark the run as unstable/failed and record the first occurrence time. Do not silently continue computing.

Required Outputs

- `t`, `Y`, `steps`, `rhs_evals`, a failure flag, and the first failure time, if any.

3. Implement Classical RK4

How to Implement It

- Implement the standard four-stage RK4 method as a separate function. Compute `k1`, `k2`, `k3`, and `k4` in sequence at each step, then update `y`.
- Standard RK4 normally makes four RHS calls per step, so increment `rhs_evals` by 4 per step.
- Keep the input/output interface consistent with EE so the convergence, stability, and cost experiments can share the same code.
- Save the full trajectory and use the same failure checks as EE.

- This RK4 implementation will also be used for fixed-step convergence, stability-region comparisons, and adaptive step doubling.

Required Outputs

- t , Y , $steps$, rhs_evals , and a failure flag.

4. Implement Implicit Euler + Newton

$$\begin{aligned} F(z) &= z - y_n - h f(t_{n+1}, z) \\ J_F(z) &= I - h J_f(t_{n+1}, z) \\ J_F(z^{(k)}) \delta^{(k)} &= -F(z^{(k)}), z^{(k+1)} = z^{(k)} + \delta^{(k)} \end{aligned}$$

How to Implement It

- Choose an initial guess for z at each time step. The simplest choice is $z^{(0)} = y_n$; a team-approved predictor may also be used.
- Within the Newton loop: (1) evaluate $F(z)$; (2) evaluate $J_F(z)$; (3) solve the linear system $J_F \delta = -F$ directly; and (4) update $z \leftarrow z + \delta$.
- Do not compute $\text{inv}(J_F) * (-F)$. Use a linear solver instead: the backslash operator (`\`) in MATLAB or `solve` in Python.
- Make the stopping criterion explicit and configurable, for example `residual_norm <= NewtonTol`. If the team also uses a correction-norm criterion, document it consistently.
- Set a maximum number of Newton iterations. If the stopping criterion is still unmet after `maxiter`, flag a Newton failure for that time step and record the step/time at which it occurred.
- Record the Newton iteration count and final residual for each time step, and accumulate `total_newton_iters`.
- If RHS/Jacobian call counts are used for cost statistics, count them consistently at the entry to the corresponding functions. Do not estimate them.

Required Outputs

- t , Y , $steps$, rhs_evals , `total_newton_iters`, per-step iteration counts (optional to save), a convergence flag, and the failure time.

5. Fixed-Step Convergence Study - Use This to Verify the Code First

$$\begin{aligned} E_{full} &= \max_i \|y_{num}(t_i) - y_{ref}(t_i)\|_2 \\ p_{obs} &= \log(E_h / E_{h/2}) / \log(2) \end{aligned}$$

How to Run It

- For each of EE, RK4, and IE, choose at least four successively halved step sizes: h , $h/2$, $h/4$, and $h/8$.
- Call the same `error_diagnostics` function for every run and compare against the shared reference solution.
- Save `E_full`. Also save `E_40`, `E_cons`, and `y_min` if possible so they can be reused directly in later experiments.
- Calculate `p_obs` from the errors at each pair of adjacent step sizes. The theoretical expectations are approximately 1 for EE, 1 for IE, and 4 for RK4.
- Plot $\log(\text{error})$ against $\log(h)$. Clearly identify all three methods, or at least the two methods required by the project.

- If the observed order is clearly incorrect, do not proceed to interpret stiffness. First check the solver, reference alignment, error calculation, and step-size range.

Required Deliverables

- Convergence table: Method, h , E_{full} , E_{40} (recommended), and observed order.
- An error-vs- h log-log convergence plot.

6. Plot the Absolute-Stability Regions of All Three Methods

How to Implement It

- The mathematics team will provide the stability functions $R(z)$ for EE, RK4, and IE. Your task is to plot the regions where $|R(z)| \leq 1$.
- Construct a grid in the complex plane, $z = x + iy$, for example using `meshgrid` to generate x and y . Evaluate $\text{abs}(R(z))$ at each grid point.
- Use `contour` or `contourf` to draw the boundary $\text{abs}(R(z)) = 1$, and clearly indicate the stable region.
- The three methods do not have to use identical axis ranges, but each range must be sufficient to show all the key boundaries. Make the legends and the $\text{Real}(z)$ and $\text{Imag}(z)$ labels clear.
- These plots should show only the absolute stability of the methods themselves. Do not mix in the Robertson eigenvalue data unless you create a separate plot with an $h\lambda$ overlay.

Required Deliverables

- EE absolute-stability region.
- RK4 absolute-stability region.
- IE absolute-stability region.

7. Perform Jacobian Eigenanalysis Along the Reference Trajectory

How to Implement It

- Loop over all reference output times with $t_{ref} > 0$. At each time, take $y_{ref}(t_i)$ and evaluate `robertson_jacobian` to obtain J_f .
- Call `eig` on J_f to obtain the three eigenvalues.
- Remove the structural zero eigenvalue using the numerical-zero threshold specified by the mathematics team. Do not simply delete the first result returned by `eig`, because its output order is not fixed.
- Sort the remaining two nonzero modes by $\text{abs}(\text{Re}(\lambda))$: label the smaller one slow and the larger one fast. Apply this rule at every time point to prevent the plotted curves from jumping when the ordering returned by `eig` changes.
- Save $\lambda_{slow}(t_i)$, $\lambda_{fast}(t_i)$, and a zero-eigenvalue diagnostic, which can help assess whether the threshold is reasonable.

Note: Do not use $t = 0$ for the stiffness ratio, because the initial state has two zero eigenvalues. Start at the first positive logarithmically spaced time.

8. Plot the Eigenvalues and Stiffness Ratio

$$S(t) = \max_{\lambda_j \neq 0} |\text{Re } \lambda_j(t)| / \min_{\lambda_j \neq 0} |\text{Re } \lambda_j(t)|$$

How to Run It

- Figure 1: Plot $|\text{Re}(\lambda_{slow})|$ and $|\text{Re}(\lambda_{fast})|$ against t , using a logarithmic time axis.

- Figure 2: Plot $S(t)$ against t , also using a logarithmic time axis.
- Depending on the range of the data, the vertical axis may also use a logarithmic scale to improve readability. Make the captions clear.
- The orders of magnitude supplied in the problem for $S(10^{-4})$, $S(10^{-2})$, and $S(40)$ are for sanity checks only. The program must calculate these values from the reference trajectory; do not hard-code them.

Required Deliverables

- The two nonzero eigenvalues vs logarithmic time.
- The stiffness ratio $S(t)$ vs logarithmic time.

9. Explicit Euler Failure / Stability Sweep

Purpose

Test the theoretical frozen-Jacobian stability estimate near $h \approx 5.9 \times 10^{-4}$, and determine when EE becomes unstable, produces negative values, or becomes clearly inaccurate in the actual nonlinear Robertson calculation.

How to Implement It

- For an initial sweep, try $h = 8e-4, 6e-4, 5e-4, 4e-4$. Add more step sizes in the critical interval based on the results.
- For each h , run to $t = 40$, or stop when a clear failure occurs and record the failure time.
- **Stable?** Check for NaN/Inf, a blow-up flag, obvious uncontrolled growth, or severe spurious oscillation. The team must agree on the decision thresholds and store them in config.
- **Non-negative?** Compute $y_{\min} = \min_{i,j} y_j(t_i)$. If $y_{\min} < 0$, record the first negativity time. The team must agree on a tolerance for whether tiny negative values caused by roundoff are permitted.
- **Accurate?** Assess E_{full} against the team-defined accuracy threshold. Do not label a solution accurate merely because it did not blow up.
- Also record E_{cons} to identify cases in which conservation holds even though the solution is inaccurate or contains negative values.

Required Deliverables

- EE failure table: h , stable?, non-negative?, E_{full} , accurate?, and the first negativity/failure time, if any.
- At least one comparison plot that clearly shows behavior near the critical step size.

10. Three Separate Diagnostics: Conservation, Positivity, and Accuracy

$$E_{\text{cons}} = \max_i |y_1(t_i) + y_2(t_i) + y_3(t_i) - 1|$$

$$E_{40} = \|y_{\text{num}}(40) - y_{\text{ref}}(40)\|_2$$

How to Implement It

- Create a separate error_diagnostics function that takes t , Y , and the reference as inputs and returns E_{full} , E_{40} , E_{cons} , $y_{\min}$, and related quantities.
- Call this same function for every key run. Do not manually recalculate the diagnostics in different scripts.
- Save conservation, positivity, and accuracy as three separate fields. A sum close to 1 does not establish that the solution is accurate or that all its components are non-negative.

Recommended Return Fields

- `E_full`, `E_40`, `E_cons`, `y_min`, `first_negative_time`, `accurate_flag`, and `nonnegative_flag`.

11. Matched-Error Comparison of Explicit Euler and Implicit Euler

What Not to Do

- Do not compare EE and IE only at the same h and then conclude that one is better.

The Correct Approach

- Use the step-sweep data you have already generated to identify one EE run and one IE run with errors `E_EE` and `E_IE` as close as possible.
- At comparable errors, compare step size, steps, RHS evaluations, IE Newton iterations, positivity, and `E_cons`.
- If RK4 is included, choose a run with a comparable error in the same way and add it as a third method.
- Discuss accuracy and stability separately. IE avoiding blow-up demonstrates strong stability; it does not imply a smaller error at the same step size.

Required Deliverables

- A matched-error comparison table, with a cost-vs-accuracy plot if needed.

12. Newton Tolerance Sweep

Purpose

Determine how accurately Newton must solve each step to be sufficient, and verify that tightening the tolerance further eventually stops producing a substantial reduction in the final error because time-discretization error dominates.

How to Implement It

- First choose a reasonable IE step size h . Keep it fixed throughout the entire tolerance sweep.
- Test `NewtonTol` = $1e-4$, $1e-6$, $1e-8$, $1e-10$, $1e-12$ in sequence. The team may choose another range, but it must span tolerances from relatively loose to sufficiently strict.
- Run IE over the full interval for each tolerance and record `y2(40)`, `E_full`, `E_cons`, total Newton iterations, and Newton convergence status.
- Identify the tolerance beyond which further tightening causes almost no change in `y2(40)` and `E_full`. This is evidence that Newton error is no longer the dominant error source.
- If Newton frequently reaches `maxiter` or fails at a particular tolerance, record that explicitly. Do not simply discard failed runs.

Required Deliverables

- Newton tolerance table: `tol`, `y2(40)`, `E_full`, `E_cons`, total Newton iterations, and convergence status.

13. Adaptive Step-Size Control

Recommended Approach: RK4 + Step Doubling

How to Implement It

- At the current point (t, y) , first take one full step of size h to obtain `y_full`.
- From the same starting point, take two consecutive steps of size $h/2$ to obtain `y_half2`.
- Since RK4 has order $p = 4$, estimate the local error from the difference, for example $\text{err} \approx ||y_{\text{half2}} - y_{\text{full}}|| / (2^p - 1)$. The team must agree on the norm and scaling.

- If $\text{err} \leq \text{tol}$, accept the step, normally using the more accurate y_{half2} , and increment `accepted_steps` by 1.
- If $\text{err} > \text{tol}$, reject the step without advancing the state, increment `rejected_steps` by 1, reduce h , and retry.
- Update h_{new} using a consistent step-size controller, such as a formula with a safety factor. Also set h_{min} , h_{max} , and maximum growth/shrinkage factors to prevent excessively abrupt changes in h .
- After each accepted step, save the current time and the actual step size h . Plot $h(t)$ at the end.
- Finally, compare the adaptive solution against the reference and report the actual E_{full} and E_{40} . Check that they are of a reasonable order of magnitude relative to the specified tolerance.

Required Records

- `accepted_steps`, `rejected_steps`, RHS evaluations, step-size history, and error-estimate history (recommended).

Required Deliverables

- An adaptive step-size plot $h(t)$, and results comparing the tolerance with the actual error against the reference.

14. Cost-vs-Accuracy Experiment

Principle

First bring the different methods to comparable accuracy, then discuss cost. Comparing which method is faster at different error levels is not meaningful.

How to Run It

- Select several accuracy targets from the convergence/sweep data, using error levels agreed upon by the team.
- For each method, find a run that achieves the target error level, then compare steps and RHS evaluations.
- For IE, also compare total Newton iterations. For adaptive methods, also compare accepted/rejected steps.
- Count actual RHS function calls. Do not replace real counters with `steps × theoretical calls per step`, especially for adaptive methods and Newton solves, where rejected steps and repeated calculations occur.
- State clearly in the report which costs are excluded, such as Jacobian assembly/factorization, plotting, or setup. Apply the same accounting convention to all methods.

Required Deliverables

- A cost-vs-accuracy table, and an error-vs-cost plot if sufficient data are available.

15. Required Report-Ready Figures

- Reference solution / species behavior. If the team report needs this figure, use the existing reference plot; there is no need to reconstruct the reference solution.
- An error-vs-step-size log-log convergence plot.
- EE absolute-stability region.
- RK4 absolute-stability region.
- IE absolute-stability region.
- The two nonzero eigenvalues vs logarithmic time.
- Stiffness ratio $S(t)$ vs logarithmic time.

- Key comparisons of EE behavior at different step sizes, including failures, against the reference.
- Adaptive step size $h(t)$.
- Any necessary matched-error / cost-vs-accuracy plots.

Requirements for all figures: Use clear titles, horizontal and vertical axis labels, units/scales, and legends. Figures must be generated and saved automatically by the code. Do not manually change data and then take screenshots.

16. Required Tables

- Convergence table: Method, h , E_{full} , and observed order. Retaining E_{40} is also recommended.
- EE failure table: h , stable?, non-negative?, E_{full} , accurate?, and the first failure/negative time, if any.
- Diagnostics table: method/run, E_{full} , E_{cons} , y_{min} , and E_{40} .
- Newton tolerance table: tol , $y_2(40)$, E_{full} , E_{cons} , Newton iterations, and convergence status.
- Cost-vs-accuracy table: method, error, steps, RHS evaluations, Newton iterations, and accepted/rejected steps, where applicable.

Table requirements: Every numerical value must be produced automatically by the code. Each row must be traceable to its specific method, step size h , tolerance, and configuration.

17. Reproducibility - Others Must Be Able to Reproduce Everything with One Command

What run_all Must Do

- Read the shared configuration.
- Load the team reference data.
- Run the convergence study.
- Generate the stability regions.
- Run the eigenvalue and stiffness analysis.
- Run the EE failure sweep.
- Run the Newton tolerance sweep.
- Run the adaptive experiment.
- Run the cost-vs-accuracy comparison.
- Save all figures in figures/ and all tables/data in results/ or tables/.

What the README Must Explain

- Required software, versions, and dependencies.
- How to run run_all.
- The responsibility of each code file.
- Where to place the reference file.
- Where output figures and tables are saved.
- The expected runtime if a full run takes a long time.
- The centralized configuration of all parameters (h , Newton tolerance, zero threshold, error norm, adaptive tolerance, and accuracy threshold), avoiding scattered hard-coded values.

18. Final Deliverables and Acceptance Criteria

Deliverable	Required Contents	Acceptance Criteria
Complete code	EE, RK4, IE + Newton, Jacobian/eigenvalue analysis, diagnostics, adaptive solver, cost counters, and run_all.	Runs from a clean environment; parameters are configurable; no manual code changes are required.
Numerical data	Raw and processed results for every experiment.	Traceable to the specific method, h, tolerance, and configuration.
Report-ready figures	Convergence, stability, eigenvalues, stiffness, EE failure, adaptive stepping, and cost.	Clear axes, units, and legends; no manual data modification.
Report-ready tables	Convergence, failure, diagnostics, Newton tolerance, and cost.	All numerical values are generated automatically by the code.
README	Run instructions, dependencies, directory structure, and output locations.	Other team members can reproduce the results by following the README.

Final Self-Check Checklist

- ☐ EE, RK4, and IE + Newton all run reliably and report explicit failure flags when they fail, rather than simply crashing.
- ☐ Convergence data for at least four successively halved step sizes have been generated, and the observed orders are reasonable.
- ☐ All three stability regions have been generated.
- ☐ The two nonzero eigenvalues and the stiffness ratio are calculated directly from the reference trajectory.
- ☐ The empirical EE sweep near $h \approx 5.9e-4$ is complete, with stability, non-negativity, and accuracy reported separately.
- ☐ E_cons, y_min, and E_full/E_40 are recorded separately.
- ☐ A matched-error comparison of EE and IE has been completed.
- ☐ The Newton tolerance sweep is complete and reports y2(40).
- ☐ The adaptive step size actually changes, and the actual error has been checked against the tolerance and reference solution.
- ☐ The cost comparison is performed at matched accuracy.
- ☐ run_all automatically generates the final figures and tables.